

# BULL II: Binding Authority to Execution

Implementation, Information Transport, Recovery, and Hardware Approval

Luis Minier  
Anharmoniclabs

**Abstract**—BULL (Blocking Unauthorized Logic Loopholes) places execution authority outside an AI agent’s generated output. The system binds an operation to host-owned permissions, an exact command or resource, admitted input bytes, and a specific security domain. It then carries that decision through process isolation, restricted brokers, and authenticated audit records. This paper documents the implementation work following the first BULL report: independent deployment authority, real resource-enforcement checks, one-shot QEMU/KVM execution, a governed local-model adapter, controlled recovery, and signed human-approval verification. The integrated candidate at 603365e recorded 489 passing tests and 21 passing subtests, five passing real-KVM cases, and authenticated host and guest receipts from the existing external collector. A separate historical Qwen run completed 40 fixed-tool actions in 707.4835 seconds, with two controlled recoveries and one credential denial. The KB2040 work produced a host verifier, durable replay protection, and diagnostic firmware; physical signing remains blocked because the inspected board had no connected secure element or enrolled signing key. Architecture drawings, measured datasets, implementation paths, and pinned reproduction procedures connect each result to the build that produced it.

**Index Terms**—Agent security, execution authorization, Linux isolation, audit transport, human approval, recovery, reproducibility.

## I. INTRODUCTION

AN agent’s proposed action becomes a security decision when software turns it into a process, a credential request, or a network operation. BULL places the enforcement boundary at that transition. A model can select work within an adapter’s exposed interface, but the trusted host supplies the identity, grants, executable binding, and runtime conditions under which that work may proceed. Authorization therefore remains attached to the operation that reaches the system, even when the proposal originated in untrusted text.

The design applies established principles of least privilege and mediation to the point where an agent’s proposal becomes an effect [1]. Related agent-security research includes CaMeL, which separates control and data flow and applies capability policies around the model [2]. BULL’s emphasis here is the joined execution lifecycle: exact request binding, admitted input, host and guest enforcement, approval consumption, and recoverable audit delivery.

The first paper introduced this execution-governance architecture and reported its initial regression, KVM, and external-audit evidence [3]. The subsequent work addressed a more demanding implementation question: how can another installation establish its own authority, verify the required

protections, recover from an interrupted run, and account for the resulting effects? Answering that question required changes across the dispatcher, host setup, guest transport, audit collector, packaging, and approval gate.

Three operational controls organize the resulting deployment: resource containment limits the work that can run; external audit preserves an authenticated reference to its history; and human approval supplies an additional credential for consequential broker operations. Their interaction is central. An isolated process still needs an exact authorization decision. A valid signature still needs current policy and a live domain. A completed process still needs trusted completion evidence. BULL implements these checks at different points in the same execution lifecycle.

This paper develops that lifecycle from request binding through recovery. It also records the practical obstacles that shaped the implementation: guest dependencies, filesystem identity, optimized-Python checks, cgroup accounting, interrupted checkpoints, and the distinction between a connected KB2040 and a functioning hardware signer. The evaluation uses the repository’s published test outcomes, benchmark summaries, run aggregates, and diagnostic records. All numerical figures are regenerated from those records.

### A. Revision and Evidence Scope

The inspected publication revision is 437de3f. Its latest integrated validation record identifies the earlier clean candidate 603365e, whose runtime source is unchanged by the subsequent publication update [4], [5]. Historical benchmarks and the governed-agent run retain their own source identities. Table I locates the main changes after Paper I [6]; Section VIII keeps their evidence separate.

## II. AUTHORITY, CONTAINMENT, AND THE SECURITY TRIAD

### A. Security Objectives and Trusted Components

BULL treats model output, workspace contents, downloaded material, and workload code as untrusted. The trusted side includes the host administrator, Linux kernel, installed runtime, authority registry, brokers, and deployment keys. The guest path adds QEMU/KVM, firmware, the guest kernel, and the guest supervisor. Collector operation and key custody are also part of the deployment’s trust assumptions [7], [8].

The confidentiality, integrity, and availability objectives use the established security vocabulary set out in NIST FIPS 199 [9]. In BULL, confidentiality depends on restricting visible files, network destinations, and credential access. Integrity depends on binding the approved request to the executed operation and preserving authenticated evidence of its outcome.

TABLE I  
IMPLEMENTATION PROGRESSION AFTER PAPER I

| Repository milestone | Implemented change                                                                           | Resulting behavior                                                                                           |
|----------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| b5059c3; PR 48       | Consequential broker approval and permission-sensitive fixture corrections.                  | Protected effects require a bound credential; tests preserve strict private-file requirements.               |
| PRs 49–50            | Unified deployment checks and independent installation authority.                            | Each operator creates private policy, integrity, and collector keys and receives distinct readiness results. |
| PRs 52–54            | Guest release recipe, source materials, and FreshClam integration.                           | Public preparation inputs and scanner dependencies can be inspected and verified.                            |
| PR 55; ee96355       | Cgroup enforcement, governed-agent acceptance, controlled recovery, and distribution checks. | Fixed tools run through production dispatch; bounded faults and package installation have recorded outcomes. |
| PR 57; fb928cb       | Custom hardware assertion provider and diagnostic firmware integration.                      | Host verification and durable consumption work in software; physical signing retains a separate gate.        |
| PRs 58–59            | 603365e integration and 437de3f evidence publication.                                        | The clean candidate’s results, setup guidance, and current hardware status are published together.           |

TABLE II  
SECURITY OBJECTIVES AND BULL MECHANISMS

| Objective       | Implemented mechanism                                                                                                                |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Confidentiality | Admitted read-only files; isolated network view; scoped egress and secret brokers; external checkpoints that omit workload payloads. |
| Integrity       | Signed policy and runtime state; exact operation binding; authenticated control frames; one-use approval; hash-linked audit records. |
| Availability    | CPU, memory, and process budgets; deadlines; cancellation; descendant cleanup; explicit recovery from bounded faults.                |

Availability depends on budgets, deadlines, cancellation, and recovery rules that prevent one workload from consuming the installation indefinitely. Table II connects these objectives to the implementation.

The deployment’s operational triad—containment, external audit, and physical approval—implements complementary parts of these objectives. Containment bounds execution. Audit makes the trusted history checkable against accepted checkpoints. Approval requires an enrolled credential before a protected effect. The current candidate records real containment and collector checks as passing, while physical approval remains blocked. This status preserves the distinction between a working software path and a complete hardware ceremony [4], [10].

### B. Exact Authorization and Delegation

The dispatcher combines an untrusted proposal with a host-constructed context. Canonicalization resolves the resource being requested; policy evaluates the actual capability and its provenance. Process launch requires `process.exec`. A file-read grant does not confer command execution, and a permitted executable does not authorize an altered argument vector. The production runtime accepts a dispatcher-minted permit that binds the checked request to the operation it will execute [8].

Delegated domains receive host-owned identities and bounded authority. A child’s capabilities and allowed egress hosts are subsets of its parent’s. Siblings remain distinct

domains while contributing to shared root behavioral history. A freeze can stop a domain after suspicious behavior without allowing the model to mint a replacement identity. Trusted provenance labels describe origin; they do not create permission.

The execution contract can be summarized as

$$\text{Execute}(r) \Rightarrow \text{Allowed}(r) \wedge \text{Bound}(r) \wedge \text{Admitted}(r) \wedge \text{Protected}(r). \quad (1)$$

Here, *bound* includes the executable and complete arguments where applicable; *admitted* identifies the approved input bytes; and *protected* denotes the mandatory runtime conditions. Equation 1 expresses the design contract. Its implementation is exercised by source regressions, parent-validated bootstrap evidence, and the recorded host and guest cases.

## III. THE EXECUTION BUILD

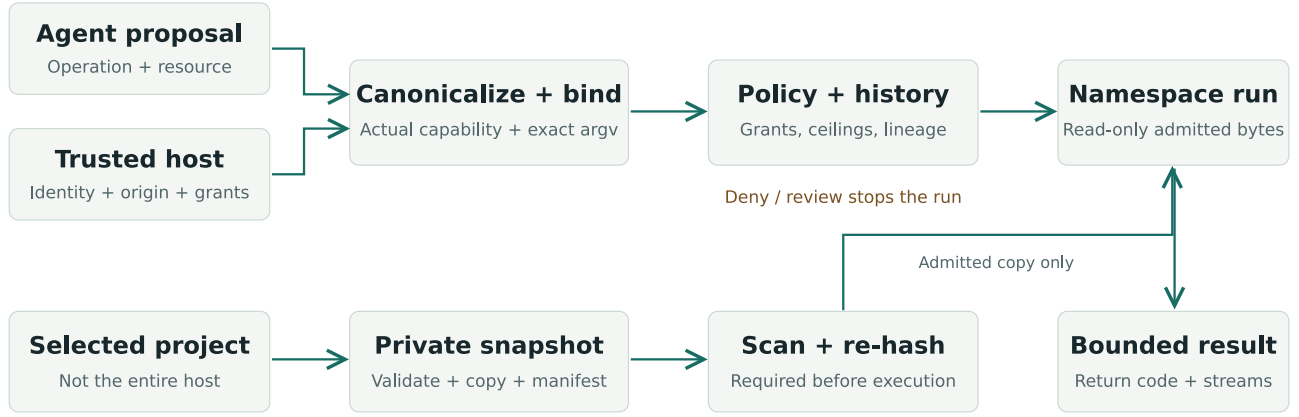
### A. From Request to Admitted Bytes

Figure 1 preserves the repository’s control and file-data drawing. The two paths meet at execution. The control path checks the operation, authority, domain, and history. The data path validates the selected project, copies it into private snapshot storage, validates the copy, scans it with ClamAV, and rehashes the admitted content. The runtime mounts the resulting snapshot rather than a changing live project [11], [12].

Admission rejects unsupported filesystem objects and changes that invalidate the copy’s identity. The manifest and rehash steps preserve a connection between the bytes inspected and the bytes exposed to the workload. The implementation also normalizes directory storage-size differences across filesystems while retaining file-content hashes and race checks. This correction made content identity portable without treating changed file data as equivalent.

The installed Python package is `bulldog-ai-firewall`, version `0.2.0rc1`, with Python 3.11 or newer. The custom hardware verifier is an optional dependency path using `cryptography>=43,<51`. Linux host provisioning, the Bulldozer guest, and Pico firmware are separate build products.

## CONTROL AND FILE DATA MEET AT EXECUTION



Command authority and filesystem admission are separate checks. Cleanup removes the private snapshot.

Fig. 1. Control and file-data paths, retained from the author’s repository drawing. Command authority and input admission converge before execution. Diagram source: [site/assets/architecture/data-flow.svg](#) [11].

Table III identifies the source modules behind the execution stages [13].

#### B. The Linux Boundary Before Execution

The namespace backend combines user, mount, PID, network, and IPC namespaces with read-only mounts, `no_new_privs`, `seccomp`, `Landlock`, and `cgroup v2` limits. The admitted `/workspace` is separate from the trusted `/bull_runtime`. Linux provides system-call filtering through `seccomp`, unprivileged access restrictions through `Landlock`, and resource accounting and limits through `cgroup v2` [14]–[16]. BULL configures these mechanisms together, including a strict default-deny syscall policy. Figure 2 shows the layers as represented in the repository [17].

The parent does not release the workload merely because the launcher started. Trusted bootstrap code installs the protections and sends nonce-bound evidence through a private pipe. The parent checks the expected `seccomp` profile, privilege state, PID-namespace setup, `Landlock`, and network conditions before releasing the pre-exec gate. The workload does not inherit the evidence descriptor. This gives the parent a checked software transition from environment setup to untrusted execution.

Resource qualification also observes behavior. The memory probe touches pages under a 64 MiB limit and requires OOM-kill evidence from kernel counters. The process probe attempts bounded forks under a four-process limit. The CPU probe performs one second of work under a 25% quota. Timeout and cancellation checks require removal of owned descendants. These finite probe settings establish whether the configured controls actually operate; production workload budgets remain separately configured [18].

#### C. One-Shot QEMU/KVM Execution

The guest backend adds a separate kernel and hypervisor boundary while retaining BULL’s admission and runtime checks

inside the guest. QEMU’s `microvm` machine type is designed for short-lived guests with a minimal device model [19]. The recorded BULL Linux x86-64 configuration uses QEMU/KVM with three read-only ext4 images: root, admitted workspace, and trusted runtime. The guest has no network device or QEMU monitor. Development 9P sharing is a separate opt-in path. Figure 3 shows the storage and bootstrap arrangement [20].

The launcher supplies pinned assets, private session channels, and an engine configuration. The guest supervisor validates the trusted engine before admitting work. The host accepts completion from authenticated session messages and audit evidence, then reaps its exact QEMU child. Each tested request starts a fresh guest; after cancellation, recovery starts another one-shot session. This lifecycle avoids carrying the cancelled guest’s memory into the next request.

### IV. INFORMATION TRANSPORT AND AUDIT

#### A. Separate Channels for Separate Authority

BULL transports file contents, execution control, workload output, and audit evidence through distinct paths. Read-only block devices carry the guest inputs. A dedicated control channel carries requests and results. A separate engine-owned audit channel carries guest evidence to the host relay. Workload output is returned as bounded data within the authenticated result [21], [22].

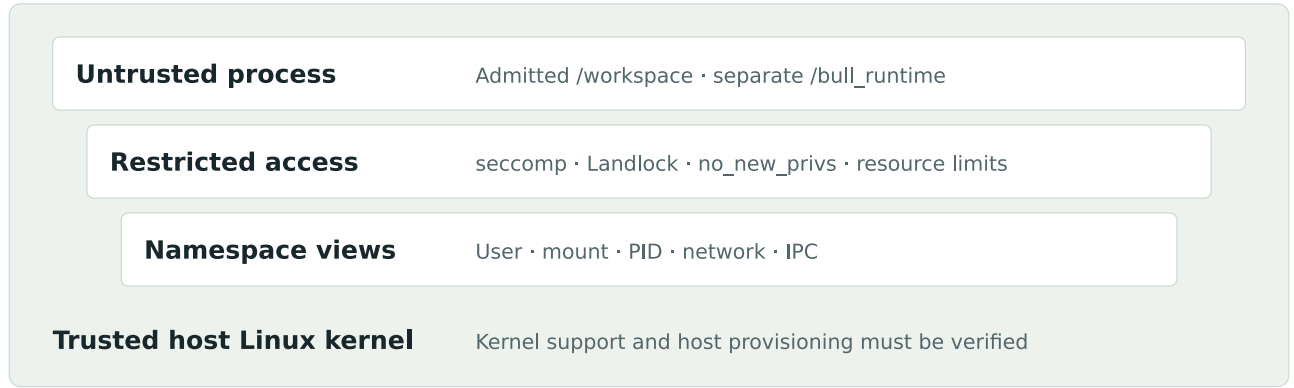
Control frames consist of a four-byte big-endian length followed by canonical JSON with version, session, sequence, kind, payload, and MAC fields. The supported kinds are `ready`, `execute`, `result`, and `error`. Frames are bounded at 65,536 bytes. Direction-separated authentication and sequence validation bind a message to the expected session and exchange. A wrong session, malformed frame, unexpected kind, or replayed sequence is rejected.

This arrangement gives the guest a narrow route for evidence without adding a general guest network interface. It also

TABLE III  
IMPLEMENTATION MAP; PATHS ARE RELATIVE TO `SRC/BULLDOG/`

| Stage                | Source entry points                                                                                                          | Responsibility                                                                       |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Authorization        | <code>canonicalizer.py</code> , <code>policy.py</code> ,<br><code>dispatcher.py</code> , <code>profiles.py</code>            | Bind the requested capability, resource, executable, arguments, and trusted context. |
| Domain authority     | <code>security_domain.py</code> ,<br><code>session_guard.py</code>                                                           | Maintain identities, delegation ceilings, lineage, and session state.                |
| File admission       | <code>snapshot.py</code> , <code>filesystem_manifest.py</code> ,<br><code>malware_scanner.py</code>                          | Copy, validate, scan, and identify the admitted bytes.                               |
| Host execution       | <code>namespace_sandbox.py</code> ,<br><code>_namespace_launcher.sh</code> ,<br><code>cgroup_scope.py</code>                 | Establish isolation, verify bootstrap evidence, release execution, and clean up.     |
| Guest execution      | <code>microvm.py</code> , <code>guest_engine.py</code> ,<br><code>microvm_protocol.py</code>                                 | Launch a one-shot guest and authenticate its session control and result.             |
| Effects and evidence | <code>secret_broker.py</code> , <code>egress_proxy.py</code> ,<br><code>approval.py</code> , <code>audit_transport.py</code> | Mediate protected effects, consume approval, and authenticate audit checkpoints.     |

#### THE NAMESPACE BACKEND SHARES THE HOST KERNEL



Current source: parent validates nonce-bound bootstrap evidence before releasing execution.

Fig. 2. The namespace execution boundary. The original repository drawing is retained, with its footer updated to the implemented parent-validated pre-execute. The host kernel remains part of the trusted base [11], [17].

prevents convincing stdout from becoming a control message. The transport authenticates the envelope; the runtime separately interprets the result under the request's execution contract.

#### B. Local Records and Remote Checkpoints

The local audit ledger stores hash-linked action and outcome records. Its trusted saved head and external checkpoints provide references against which the presented history can be checked. The direct HTTPS checkpoint contains the session identifier, sequence number, current head, previous checkpoint hash, and authentication tag. Prompts, secrets, full local records, and workload payloads are omitted from this upstream message [22].

The guest route adds a relay. The engine-owned audit port carries the full guest audit record to the trusted host, which retains local evidence and sends the smaller checkpoint upstream. The relay returns success only after authenticating the upstream acknowledgement. The external collector master key

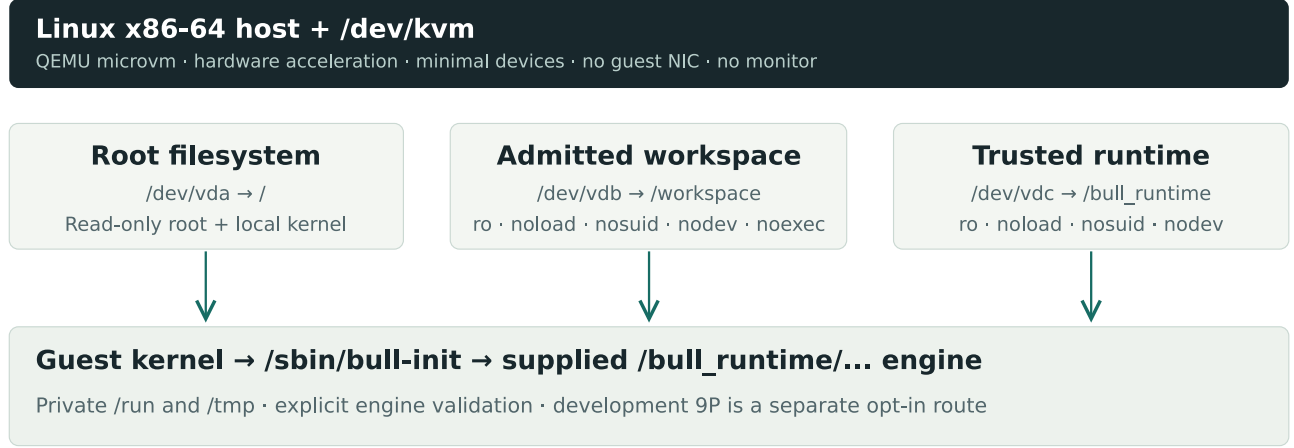
remains host-side; guest relay authority is separate. Figure 4 retains the repository's audit-flow drawing [22], [23].

HTTPS protects transport and HMAC-SHA256 authenticates checkpoint messages with deployment-owned authority. HMAC uses the keyed-hash construction specified in RFC 2104 [24]; the BULL transport defines the fields and acknowledgement checks [22]. Acceptance requires an authenticated reply for the exact session, sequence, and head, with HTTP 200 and no redirect. A reachable endpoint alone does not satisfy this condition.

#### C. Checkpoint Failure and Reconciliation

A checkpoint failure stops subsequent appends until the retained evidence is explicitly reconciled. This ordering became operationally significant during the external-collector work. Cloudflare rejected the default Python client signature before the Worker accepted the request, leaving a local record without its matching remote checkpoint. The implementation introduced the explicit `BULL-AuditAnchor/1` client identifier.

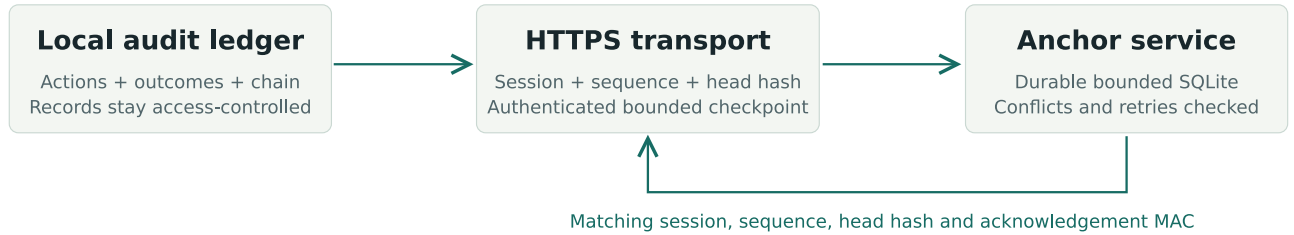
## LOCAL HOST ASSETS → READ-ONLY VIRTIO BLOCK → GUEST



Current record: one-shot KVM and external receipts passed; persistent VM reuse remains unverified.

Fig. 3. Read-only guest inputs and bootstrap, from the repository’s MicroVM drawing. The status annotation reflects recorded one-shot KVM and external-receipt validation. Original and updated SVGs are included in the companion source [11], [20].

## EVIDENCE LEAVES THROUGH A DIFFERENT CHANNEL



The direct HTTPS transport omits the full record. The one-shot guest relay requires a trusted, acknowledged bootstrap.

Fig. 4. Audit transport and checkpoint acknowledgement, adapted from the repository drawing. The SQLite service shown is the reference collector; the deployed Cloudflare service uses D1. The guest-bootstrap annotation has been updated to the current implementation [11], [22], [23].

The operator then verified the local record, reconciled the remote checkpoint, verified the combined state, and completed a fresh authenticated append [25].

Reconciliation retries evidence delivery. It does not rerun the action that produced the record. That distinction preserves the original event when transport fails after an effect. The existing live collector later passed authenticated host and guest receipt checks for the integrated candidate. A separate audit-worker build failed, and an earlier preview that returned HTTP 409 at its first checkpoint remained unpromoted [4], [26]. These are distinct deployment outcomes.

## V. APPROVAL BOUND TO A SINGLE EFFECT

### A. Policy, Signature, Consumption, and Execution

Human approval is an additional condition on an already-permitted operation. The implemented consequential paths are credential retrieval and scoped network-broker requests. Exact

HTTPS GET/HEAD URLs listed in the signed routine policy form a network exception; credential access remains approval-gated [27].

The request binds the operation, target, parameters, actor, domain, trusted context, session, grants, provenance, and policy digest. Executable identity and the complete argument vector belong to the binding where relevant. A changed action requires a new request. The approval lifetime is bounded, and the credential must be enabled in the installation’s enrollment state.

After verifying the signature, the gate atomically consumes the request and advances the custom provider’s per-key counter in SQLite. It then delivers the required approval audit event, rechecks current policy and domain authority, and calls the broker. Figure 5 shows the state order. If audit fails after consumption, the approval stays spent. If the broker raises after attempting its effect, the outcome is recorded as uncertain [28].

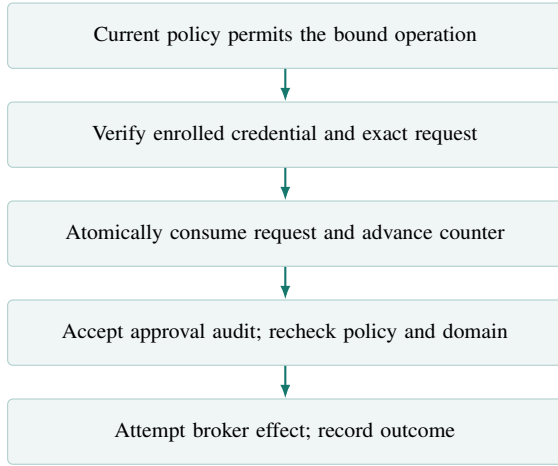


Fig. 5. Ordering in the durable approval gate. Consumption precedes the effect and remains committed when a later stage fails. This is the implemented software order [28].

The resulting property is at-most-once authorization consumption. SQLite’s atomic-commit model concerns the database transaction and its storage assumptions [29]. BULL uses that boundary to keep request consumption and counter advancement together. The local transaction does not create an atomic transaction with an external service. An uncertain remote effect requires an explicit recovery decision, and a consumed signature cannot be recycled to try it again.

### B. Two Credential Verification Paths

OpenSSH documents `ecdsa-sk` and `ed25519-sk` credentials, application strings, and separate touch and user-verification requirements [30]. BULL’s provider uses the `ssh:bull-approval` application and checks the signed presence and verification flags [27]. The custom provider verifies fixed-width BULL P-256 assertions, including the enrolled identity, request binding, expiry, and counter. Synthetic credentials exercise the software paths in CI; the physical ceremony is a separate deployment check [31].

FIDO CTAP 2.1 distinguishes evidence of user presence from verification of the user’s identity [32]. That distinction provides a useful reference for the approval design: a gesture, a verified credential, and the exact action presented for approval are separate parts of the ceremony. BULL-AUTH-1 is the custom protocol evaluated in this paper.

For the custom provider, counters follow the public-key fingerprint across enrollment aliases. Request consumption and counter advancement share the same durable transaction. Skipped counters are acceptable because a failed delivery may consume a device counter value; stale or repeated counters are rejected. Signed denial is terminal and audited separately.

## VI. THE KB2040 SIGNING HURDLE

### A. What the Bring-Up Established

The KB2040 work exposed the gap between USB device detection and cryptographic authority. The connected board reported `BULL-DIAG-3` and `approval_available=false`.

TABLE IV  
KB2040 CONNECTION PLAN FROM THE REPOSITORY

| Pin            | Specified connection                                |
|----------------|-----------------------------------------------------|
| 3V; GND        | Compatible secure-element supply and common ground. |
| GPIO12; GPIO13 | Secure-element SDA and SCL.                         |
| D4 / GPIO4     | Normally-open YES button to ground.                 |
| D5 / GPIO5     | Normally-open NO button to ground.                  |
| GPIO17         | Onboard NeoPixel status indication.                 |

Diagnostics recorded no valid ATECC wake response, zero successful four-byte reads, and high SDA/SCL levels at boot. The operator confirmed the physical configuration: only the KB2040 was present, with no secure-element breakout attached. There was therefore no hardware signing key to generate, provision, or enroll [10].

The diagnostic firmware exposes vendor-defined HID status, reads button levels, and probes the bus. Its build target is `bull_hardware_diagnostic`. It contains no private signing key and exposes no keyboard, mass-storage interface, CDC console, or REPL. Its diagnostic response cannot satisfy the approval verifier. The host integration consequently preserves the missing credential as a blocked approval result.

The completed software work includes 24 stateless verifier cases, 23 durable-gate cases, C button-state-machine tests, and a Pico SDK 2.2.0 firmware build [4]. The firmware toolchain uses ARM GCC/newlib and picotool 2.2.0; an explicit `pico_unique_id` link dependency corrected the diagnostic build. The physical click test was cancelled, so the two-/three-click diagnostic gesture remains unwitnessed on the board.

### B. Wiring and Presence State

The intended design places a non-exportable P-256 key inside a validated secure element and uses the controller to request a signature after physical input. Table IV gives the repository’s connection plan. Adafruit’s board documentation identifies the power, D4/D5, SDA/D12, and SCL/D13 connections [33]. The original external-button state machine requires both buttons to be released for 30 ms after a request, followed by YES debounce and a 750 ms continuous hold. Stable NO denies; simultaneous presses, expiry, or disconnect cancel [10].

The later diagnostic interface implements two BOOT clicks for YES and three for NO. It applies 30 ms edge debounce and a 600 ms quiet window after the second release; a third click yields NO. An abandoned single click, a 1.5-second hold, request expiry, or disconnect cancels. Idle clicks do not carry into a later request. This diagnostic interface has not replaced the separate-button production acceptance specification.

### C. The Signed Message

BULL-AUTH-1 uses fixed-width, big-endian fields with no padding. The request contains the protocol prefix, type, eight 32-byte fields, and two unsigned 64-bit timestamps. It binds request ID, session, complete action, operation, resource, parameters, policy, and nonce. Its lifetime is at most 30 seconds. SHA-256 of the encoded request supplies the request digest [31].

The assertion contains the protocol prefix, decision, five 32-byte fields, a 32-bit counter, and a 64-byte P-256 signature in  $r||s$  form. The fields bind enrolled device identity, request ID, request digest, session, and nonce. ECDSA-SHA256 signs the bytes preceding the signature. The implementation's structs encode a 284-byte request and a 240-byte assertion within a 512-byte packet ceiling. These are protocol sizes derived from the codec.

The next physical milestone is a verified signing path: identify the actual secure element and its configuration, test its key generation and signature operations, enroll the public key under operator supervision, implement authenticated host requests and bounded signing HID transport, and witness the approval and rejection cases. A secure element addresses key extraction; trustworthy button enforcement also depends on controller firmware and the operator's view of the requested action. Those trust questions remain explicit in the hardware design.

## VII. ENGINEERING METHOD AND CORRECTIVE WORK

### A. Test the Effect and the Failure Boundary

The repository's method starts with a concrete invariant and an observable effect. A permitted operation supplies the positive control; a changed grant, argument, origin, or runtime condition supplies a forbidden variation. Source identity, inputs, and deployment authority are recorded before comparing the result. Interruption cases then test what happens between admission, execution, evidence delivery, and completion [7], [34].

This approach shaped the standalone adversarial harness. A denial can return a normal `ExecutionResult` with `executed=false`; an exception is not the only correct refusal. The harness was corrected to inspect that result, use real authorized argument vectors, and retain envelope hop history. Its 49 probes exercise source/API behavior. The live kernel, collector, and guest paths have separate acceptance procedures [6].

Table V records implementation changes driven by failed or misleading checks [5], [12], [35]. The common thread is preservation of the control while correcting the setup, measurement, or dependency that prevented it from working as intended.

### B. Recovery with an Explicit Interruption Point

Seven controlled cases cover ENOSPC and EIO at append and head-write boundaries, ledger-budget refusal, worker exit with a pending plan, and outage/restart of a disposable TLS collector with retained SQLite state [36]. If append fails before writing a record, the prior history remains valid. If the record is written but its trusted head cannot advance, subsequent appends stop and retain the unresolved state.

Recovery uses the interruption point to decide what can proceed. A checkpoint retry delivers evidence already produced. A pending read-only action may be rerun under the adapter's contract. A completed action is retained as completed. This is why the current governed adapter exposes only fixed `inspect` and `checksum` operations: an incomplete read can be repeated without introducing a new remote mutation.

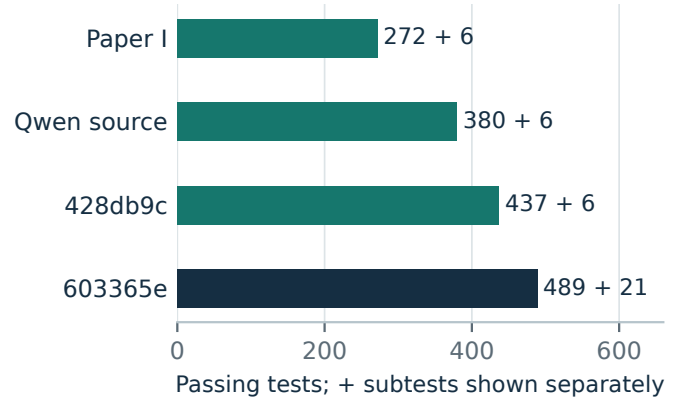


Fig. 6. Passing tests at four recorded milestones. Bar lengths count tests only; labels show tests plus subtests. These are distinct suites rather than cumulative totals or a security-coverage score.

### C. Finite Models of the State Transitions

TLA+ specifies system behavior as state transitions, and TLC checks configured models against their properties; Lamport develops the language and method in *Specifying Systems* [37]. BULL's specifications express selected runtime, session/audit, and approval invariants. The recorded models contain 405, 57, and 112 distinct states, respectively. All three parse/model gates passed for the integrated candidate [4], [38]. The runtime model checks the relationship between authority, required protections, and execution. The session/audit model checks ordering and acknowledgement behavior. The approval model checks bounded pending/uncertain states and the restriction on effects without approval or after repeated use. These finite models complement the implementation tests by making the allowed transitions explicit.

## VIII. EVALUATION AND RECORDED DATASETS

### A. Dataset Identity

The evaluation data are defensive regression outcomes, measured durations, source manifests, run aggregates, and device diagnostics. Table VI identifies each dataset and its experimental unit. The companion archive preserves the public records and CSV exports used to generate the numerical figures. Historical source manifests are retained where a run included uncommitted changes.

Figure 6 shows the reported suite sizes at four milestones. Paper I recorded 272 passing tests plus six subtests. The historical governed-agent source recorded 380 plus six; the earlier clean candidate 428db9c recorded 437 plus six; and the integrated candidate recorded 489 plus 21 [3], [4], [26], [40]. These counts describe the recorded suites as the implementation changed.

### B. Selected Defensive Corpus

The September 20 operator benchmark ran eleven selected defensive scenarios three times each, recording 33 passes and zero failures. The cases exercise path traversal, command and argument substitution, external-origin credential access, broker

TABLE V  
RECORDED HURDLES AND THE CORRECTIONS THEY PRODUCED

| Observed hurdle                    | Correction                                                                                                                        | Implementation consequence                                                                                      |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Guest configuration and boot       | Permit digits in literal <code>DEV_9P</code> keys; pin qboot and non-ACPI setup; correct AMD SSBD feature dependencies.           | Supported inputs and CPU configuration work while literal-only parsing and mitigation enforcement remain.       |
| Guest scanner dependencies         | Supply Bash, ClamAV, verified offline databases, and the correct database path; raise the historical scanner ceiling to 1536 MiB. | Admission can load the full databases; its budget remains separate from the 512 MiB workload limit in that run. |
| Filesystem-dependent manifests     | Normalize directory storage sizes during content equality while retaining file hashes and race checks.                            | The same admitted content can be compared across filesystems.                                                   |
| Unsafe ambient file modes          | Set private fixture modes explicitly and add nine unsafe-mode rejection cases.                                                    | Codespaces tests stop depending on the ambient umask; runtime checks remain strict.                             |
| Uncharged memory allocation        | Touch allocated pages and require OOM counters and SIGKILL evidence.                                                              | The probe observes charged memory and actual limit enforcement.                                                 |
| Assertions removed by optimization | Replace security-relevant <code>assert</code> checks with explicit errors.                                                        | Eight invalid cases and one valid control pass across four Python modes, for 36 software cases.                 |
| UID-wide process pressure          | Place the workload in its own cgroup and exercise bounded forks there.                                                            | Unrelated desktop processes no longer determine the workload’s process budget.                                  |
| Transport and path failures        | Preserve scanner-timeout and lost-relay failures; use shorter private Unix-socket paths.                                          | Setup and delivery failures retain their actual outcomes in the validation record.                              |

TABLE VI  
DATASETS USED IN THIS PAPER; EACH RETAINS ITS OWN SOURCE IDENTITY

| Dataset                 | Unit and size                          | Recorded source and conditions                                                          |
|-------------------------|----------------------------------------|-----------------------------------------------------------------------------------------|
| Defensive corpus        | 11 scenarios $\times$ 3 invocations    | cd461ae; end-to-end pytest timing and peak RSS summaries [39].                          |
| Direct hot paths        | 3 loops $\times$ 500 iterations        | Same benchmark source; one primitive and two accumulated-state multi-agent loops [39].  |
| Historical KVM timing   | 9 overlapping intervals; 1 allowed run | d218e688 plus recorded dirty-tree hash; local disposable TLS collector [12].            |
| Historical Qwen adapter | 40 completed actions; 130 records      | 974eb17 plus exact source manifest; host namespace sandbox and existing collector [26]. |
| Integrated candidate    | 489 tests; 21 subtests; 5 KVM cases    | Clean 603365e; source, deployment, packaging, and external-receipt checks [4].          |
| Hardware inspection     | Read-only HID diagnostic record        | BULL-DIAG-3; no secure element; physical gesture test cancelled [10].                   |

access under a sandbox verdict, private/metadata SSRF targets, Unix peer identity, prompt injection, forged policy verdicts, and approval-token replay [39]. Figure 7 plots the recorded mean and observed range for every scenario.

The host was CachyOS Linux 7.2.6-1-cachyos x86-64, Python 3.14.7, an AMD Ryzen 5 7430U with 12 logical CPUs, and approximately 11 GiB of memory. Scenario means ranged from 292.603 to 376.640 ms; recorded peak RSS values ranged from 44.668 to 46.086 MiB. The clean full regression invocation took 4.587 seconds. These measurements describe the test procedure, including its framework overhead. The approval-token replay case belongs to the earlier multi-agent corpus and predates the custom hardware provider.

### C. In-Process Measurements

The same benchmark recorded three 500-iteration loops, each with 500 expected matches. Traversal rejection averaged 4.897 microseconds per operation. Prompt-injection denial averaged 66,559.856 microseconds, and benign multi-agent allow

averaged 438,559.052 microseconds. The reported throughputs were 204,187.31, 15.02, and 2.28 operations per second, respectively [39].

Figure 8 exposes the scale difference between the tight rejection primitive and the stateful orchestration paths. The two multi-agent loops reuse one `MultiAgentSystem`, so their results include growing trace and honeypot state. The published data do not isolate individual coordinator, canary, policy, auditor, or verifier costs. Component-level optimization therefore requires additional instrumentation rather than allocation of the existing totals.

### D. Historical KVM Timing

The historical allowed KVM run recorded nine intervals, shown in Fig. 9. QEMU start to dispatcher-ready took 12,157.320 ms; launcher to ready took 12,282.303 ms. Dispatch, including admission and scanning, took 9,165.705 ms. The fixture body reported 0.476 ms. Startup and admission consequently dominate this small fixture’s useful work [12].

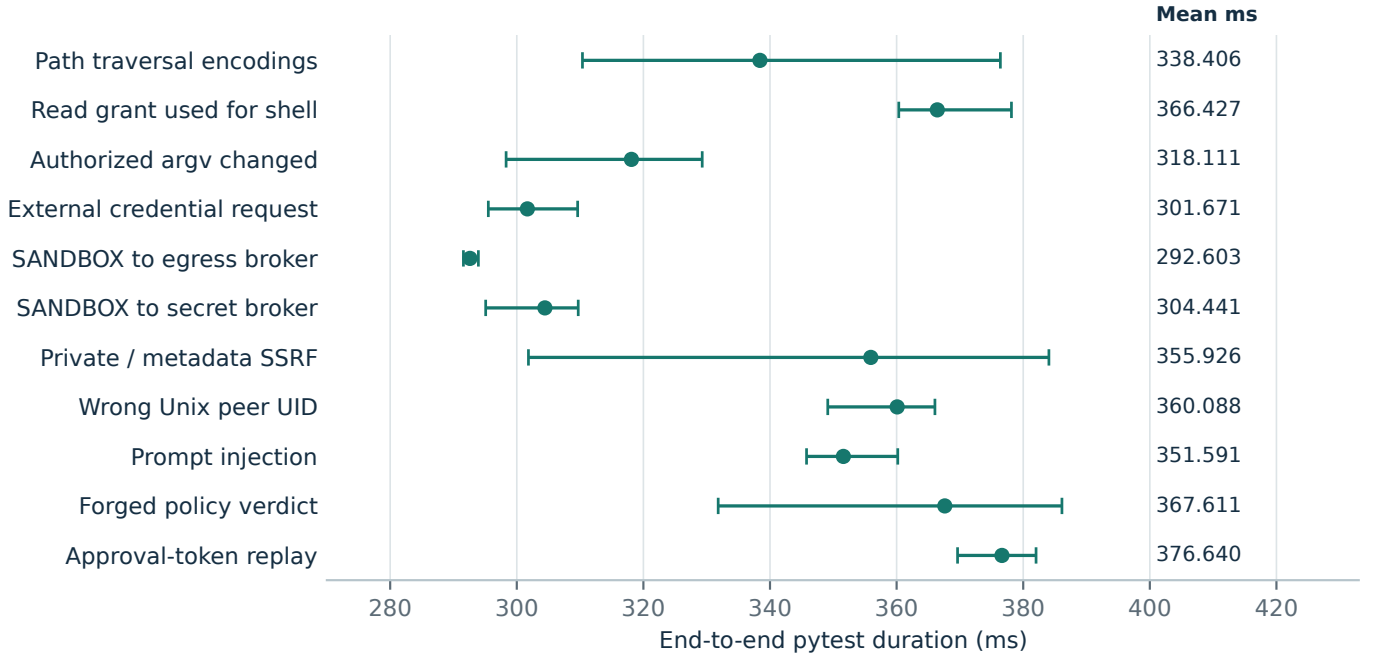


Fig. 7. The recorded defensive corpus at `cd461ae`. Dots show means; whiskers show the observed minimum and maximum of three invocations per row. Each scenario passed 3/3. Durations include Python, pytest, fixtures, and process startup. Source: `data/attack_measurements.csv` [39].

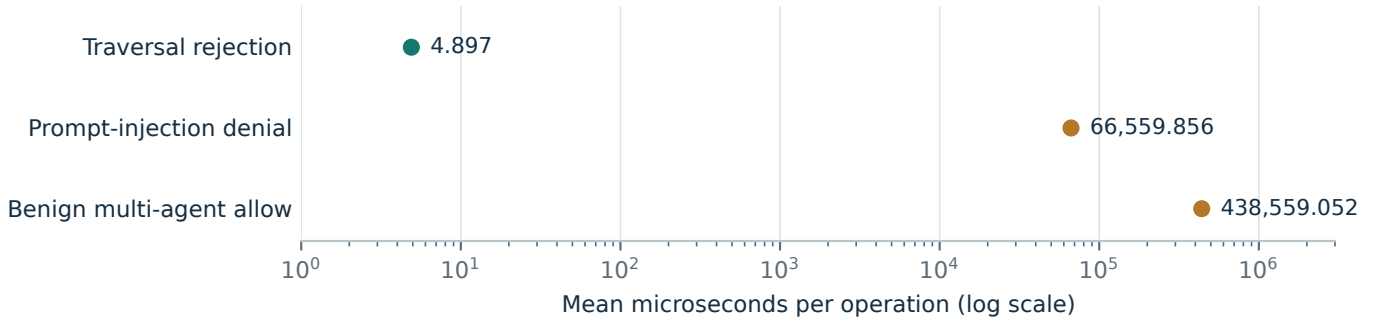


Fig. 8. Recorded in-process means, with a logarithmic horizontal scale. The teal point is a traversal primitive; amber points are multi-agent loops that reuse one system for 500 iterations and accumulate trace/honeytoken state. The differing loop constructions are retained in the interpretation [39].

The 99.492 ms pre-exec-to-reap interval includes interpreter startup, execution, and process reaping. Completion audit acknowledgement took 33.714 ms, and result/audit work outside dispatch took 34.355 ms. The report identifies a dirty source tree based on `d218e688`, an AMD-native SSBD profile, a disposable authenticated local TLS collector, a 1536 MiB scanner ceiling, and a separate 512 MiB workload address-space limit. Later candidate validation records case outcomes without publishing a replacement timing series.

#### E. Integrated Candidate Results

At clean `603365e`, the source suite recorded 489 passing tests and 21 passing subtests with no failures or skips. Six GitHub validation workflows completed successfully, covering security regressions, MicroVM host regressions, formal invariants, guest configuration, FreshClam integration, and Pages deployment. The separate Cloudflare audit-worker build failure remains recorded alongside those results [4].

The five real-KVM cases passed their respective acceptance conditions (Table VII). The fixed-tool host adapter separately executed `inspect` and `checksum` in the sandbox; an empty-grants request returned `DENY` with `executed=false`. A fresh guest after cancellation passed as a new session. Packaging built a wheel from the source distribution and installed it in a fresh environment outside the checkout.

The machine-readable deployment record contains thirteen passing gates and one blocked gate: physical hardware approval. Passing gates include strict host checks, cgroup delegation, guest assets, current-candidate KVM, local TLS, configuration, source identity, software approval protocol, and external collector receipts with both host and guest paths. The record qualifies the software candidate and leaves physical approval blocked [4].

#### F. The Forty-Action Governed Model Run

The historical adapter used a locally installed `gwen2.5:1.5b-instruct` model to choose between fixed

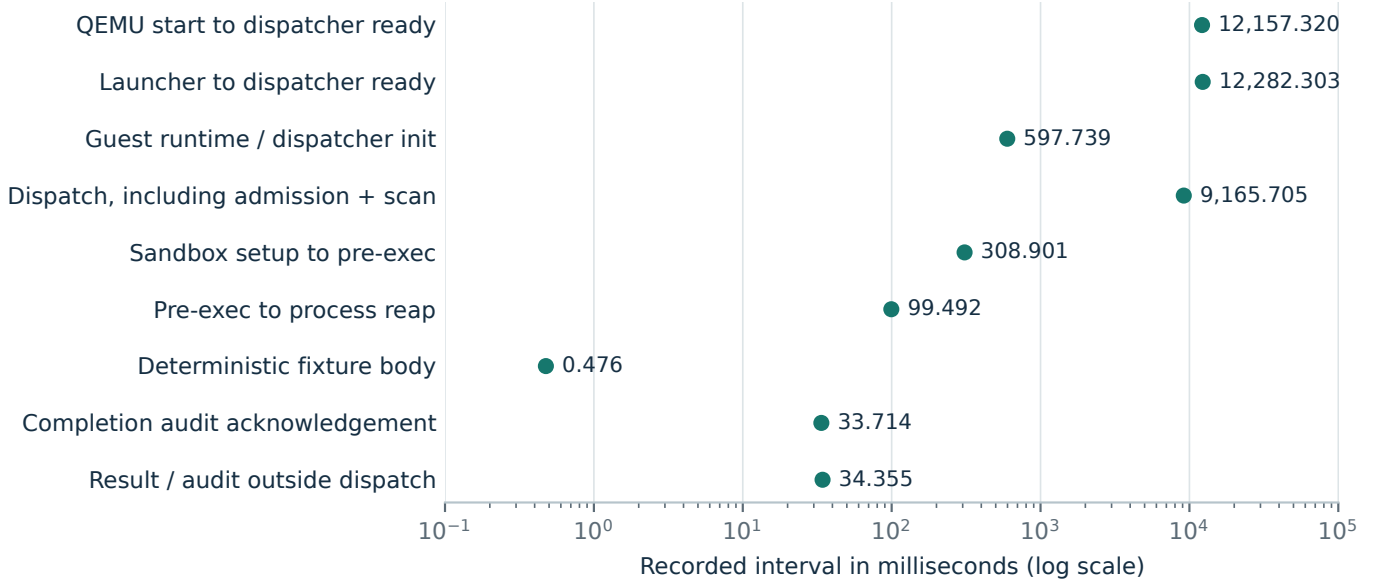


Fig. 9. Nine intervals from one historical allowed KVM run. Values are milliseconds on a logarithmic scale. Intervals overlap and must not be added. The figure retains the original run’s source identity and is not a timing measurement of 603365e [12].

TABLE VII  
REAL-KVM ACCEPTANCE AT THE INTEGRATED CANDIDATE

| Case               | Required and recorded outcome                                             |
|--------------------|---------------------------------------------------------------------------|
| Allowed            | Authorized work reaches execution and returns a trusted completed result. |
| Denied             | Refusal occurs before workload execution.                                 |
| Timeout            | No successful completion is accepted; owned work is cleaned up.           |
| Cancellation       | The exact QEMU child is reaped; interrupted work remains incomplete.      |
| Missing protection | Required protection failure stops admission.                              |

inspect and checksum tools on a known fixture. Trusted host code supplied the tool bodies, paths, argument vectors, identity, and grants. Model output was validated as untrusted JSON, and the prompt deliberately exercised both tools [26], [41].

The run completed 40 actions in 707.483500783 seconds, recorded two recovery events, one credential denial, and 130 ledger records, and finished with no pending plan. Its backend was the host `NamespaceSandbox` through `ProductionDispatcher` and `ProductionRuntime`; the supervisor and Ollama service were trusted host processes. These aggregates describe a host adapter run.

The first worker saved a plan and exited with code 75 before dispatch. The next invocation recovered that plan, performed the read-only tool, and exited with code 76 before recording completion. A third invocation recovered again and finished. Committed steps were preserved, while an incomplete read could execute again. A fresh deployment from reconstructed historical source separately completed two actions with two recoveries and one credential denial in 36.60895827 seconds [26].

Listing 1. Source validation from a fresh checkout

```
git clone \
  https://github.com/Anharmoniclabs/BULL.git \
  BULL-paper2
cd BULL-paper2
BULL_REF=603365edd0164418a06ec0ee46250a2922b284bf
git checkout --detach "$BULL_REF"
git rev-parse HEAD
git status --porcelain
python3 -m venv .venv
.venv/bin/python -m pip install \
  -e '[test,hardware]'
.venv/bin/python tools/release_check.py \
  --tla-jar /ABS/PATH/tla2tools-1.7.4.jar \
  --output /tmp/bull-source-01
```

The historical bundle also includes five separate KVM cases using a local disposable TLS collector; the agent itself used the existing live collector. Its source is 974eb17 plus an exact dirty-source manifest, with a 380-test and six-subtest regression result. The later 603365e evidence does not rerun this forty-action experiment.

## IX. REPRODUCING THE BUILD AND THE EVIDENCE

### A. Pin the Candidate and Record the Environment

Reproduction starts from the tested candidate rather than a moving branch. Listing 1 creates a fresh checkout and invokes the repository’s source release check. Java and the pinned TLA+ 1.7.4 tools are required; the repository records the expected jar digest. Dependency ranges in the test extra should be preserved with the resolved installed versions [18].

Source validation and deployment qualification are separate commands. On a supported Linux host, `tools/host_setup.py` delegates a dedicated cgroup subtree and can grant KVM access, then enters an unprivileged shell.

Listing 2. Initialize a new deployment after host delegation

```
BULL_STATE="$HOME/.local/share/bull/paper2-01"
.venv/bin/python \
  tools/deployment_setup.py init \
    --state "$BULL_STATE" --project-root "$PWD" \
    --cgroup-parent "$BULL_CGROUP_PARENT"
```

Listing 3. One-shot guest acceptance with verified local assets

```
PYTHONPATH=src .venv/bin/python \
  microvm/integration.py \
    --kernel /ABS/PRIVATE/bzImage \
    --rootfs /ABS/PRIVATE/rootfs.ext4 \
    --firmware /ABS/PRIVATE/qboot.rom \
    --output /tmp/bull-kvm-01 \
    --case all --cpu-profile host
```

Host checks require the documented namespace, seccomp, Landlock, scanner, and utility dependencies. KVM and USB credential access remain independent prerequisites.

### B. Create Independent Installation Authority

The deployment initializer creates private state outside the admitted project and Git checkout. It generates independent integrity, policy, and collector keys, together with installation and session identifiers. Directories use mode 0700 and private files use 0600. An existing state path is refused, preserving its keys and history [18].

The operator then provisions a compatible HTTPS collector using that installation’s authority, configures the endpoint, and supplies verified guest assets. The linked Worker/DI procedure provides the reference deployment [23]. Running `tools/deployment_check.py` with the private deployment path, TLA+ jar, and a new short evidence directory produces separate gate results. Missing assets or physical credentials remain `BLOCKED`; the report, source identity, logs, and hashes should be retained together.

### C. Guest and Model Acceptance

The guest release supplies documented assets and a public preparation recipe. After checking their manifests, `microvm/integration.py` exercises the five cases. Listing 3 uses a disposable local TLS collector by default. The `-external-url` and `-external-key-file` options select an operator’s collector for a separate external-receipt run [20].

For model acceptance, the deployment project root is `examples/governed-agent`, with its read/process ceiling, signed state, delegated resources, scanner, and collector. Ollama must already have the selected model installed. The acceptance tool’s `-enter-coordinator` option runs the bounded procedure under the configured resource hierarchy [41].

Acceptance requires the expected tools, outputs, recoveries, sandbox evidence, credential denial, and empty pending-plan state. The ledger can be checked with `tools/audit_governed_agent.py`, using an independently retained expected head where available. Record the source,

Listing 4. A new governed-agent acceptance run

```
.venv/bin/python \
  tools/check_governed_agent.py \
    --deployment "$BULL_AGENT_STATE" \
    --output /tmp/bull-agent-01 \
    --model qwen2.5:1.5b-instruct \
    --steps 40 --interval 5 --max-seconds 7200 \
    --enter-coordinator
```

model digest, fixture identity, scanner databases, dependencies, and host configuration. Fresh sessions and installation keys intentionally differ across operators.

### D. Distribution and Paper Reconstruction

The `guest-2026-09-22` source-materials release reconstructs pinned BULL, Buildroot, and qboot inputs alongside archives, patches, licenses, signed databases, and resolved configurations. The recorded review checked 67 package/version entries against the original build log. Offline preparation and verification of 57 source archives passed with external networking unavailable. A full offline recompile is still unrun [42].

The paper companion includes this native LaTeX manuscript, the supplied Paper I LaTeX input, four repository technical drawings and their original SVGs, data exports, unchanged public evidence records, plotting code, and a checksum manifest. Its figures can be regenerated without fetching new measurements. The retained source identities and reproduction commands support inspection of both the paper and the implementation.

## X. RELATED WORK AND POSITIONING

### A. Protection Principles and Agent Control

Saltzer and Schroeder’s protection principles place authority checks, restricted privilege, and conservative defaults at the center of system design [1]. The architectural connection to BULL is the placement of those checks in trusted execution paths. The model supplies a proposal; host-owned context and enforcement determine which effects can occur. Rechecking authority before a broker effect also addresses the lifetime of a decision when policy or domain state can change.

CaMeL applies a protective layer around an LLM, extracting control and data flows from a trusted query and using capabilities to constrain information movement [2]. Its focus makes it directly relevant to BULL’s separation of model output from authority. BULL’s work in this paper is concentrated at the execution and deployment boundary: command and argument binding, snapshot admission, Linux and guest protections, durable approval consumption, and authenticated completion. These are architectural comparisons; the two systems have not been evaluated on a shared workload in the reported experiments.

### B. Isolation and Audit Evidence

Firecracker studies a specialized virtual-machine monitor for serverless execution [43]. QEMU’s `microvm` documentation identifies a related minimal machine model for short-lived

guests [19]. BULL uses the QEMU/KVM path and adds its own admission, authority, transport, and completion rules. The historical timing figure measures that BULL configuration. Firecracker’s performance results remain evidence about Firecracker.

Schneier and Kelsey study cryptographic protection of audit logs on a machine that may later be compromised [44]. Their work establishes a broader research context for preserving trustworthy history. BULL’s implemented approach combines a local hash chain with authenticated external checkpoints and explicit reconciliation. The result studied here is continuity between a recorded event and its accepted checkpoint under the tested interruptions. The paper’s trust model and retained checkpoint authority determine what that history can establish.

### C. Evaluation and the Contribution of This Build

AgentDojo provides an extensible environment for evaluating tool-using agents, prompt-injection attacks, and defenses [45]. It is relevant to a subsequent evaluation because useful task completion and resistance to adversarial tool output can be examined together. Applying it to BULL would require a declared mapping from benchmark tools to capabilities, resources, and effect adapters. The existing defensive corpus and forty-action run retain their own acceptance criteria; no AgentDojo score is reported.

Taken together, these references locate the contribution in the integration and recorded behavior of BULL’s execution lifecycle. Established protection mechanisms supply the underlying controls. BULL binds them to a request and follows the resulting operation through admission, execution, approval, evidence delivery, and bounded recovery. Repository records support the claims about this build; independent literature and primary specifications supply the technical foundations and comparison points.

## XI. DISCUSSION AND LIMITS

The demonstrated result is an integrated execution lifecycle for specific host and one-shot guest paths. Exact authority, admitted input, required protection, and authenticated evidence are joined in the implementation and exercised in the recorded cases. The data support that statement at identified revisions and under documented operating conditions. They also expose where the next engineering work belongs.

The first boundary is the trusted base. A compromised administrator, kernel, runtime, or deployment key can act outside the assumptions of the checked path. A permitted operation can still be harmful within its granted authority, and a broad combination of read and send permissions can permit disclosure. Hashes identify bytes and scanners apply their detection rules; neither establishes the semantics of arbitrary future behavior. The development multi-agent callable executor also remains distinct from a production adapter whose effects pass through the dispatcher.

The second boundary is evidence interpretation. The selected defensive corpus does not measure a broad malicious-effect rate, benign-task false-denial rate, or comparative performance against another system. The hot-path constructions differ, the

KVM timing has one observation, and the historical model record contains aggregates rather than per-action traces. Public summaries omit private deployment keys and raw ledgers, which limits independent authentication of historical receipts. No new live deployment experiment was performed for this manuscript revision.

The third boundary is recovery and availability. The recorded faults are injected storage errors, controlled process exits, and a disposable collector restart. They do not establish host power-loss durability or multi-day reliability. Persistent guests, writable workspace promotion, publishing, email, and protected deletion require their own adapters and effect-specific recovery rules. Authenticated checkpoint acceptance also does not establish independent retention or undo an external side effect.

The remaining hardware work is substantive. The current KB2040 has no connected secure element or enrolled signing key. Adding a non-exportable key would address extraction, while the RP2040 controller still lacks the authenticated firmware trust of a hardened secure-boot design. A trusted display is also absent. Physical presence, signed request binding, firmware integrity, and what the operator sees require separate validation. The prototype is not a certified authenticator.

Finally, the project records owner-operated evidence rather than an independent security assessment. Its source-materials release establishes inspectable preparation inputs; two independent byte-identical guest builds have not been demonstrated. These limits define concrete follow-up work: complete and witness the physical signing ceremony, evaluate new adapters by their actual effects, measure legitimate completion alongside prohibited effects, and test longer-lived recovery under explicitly declared failures.

## XII. CONCLUSION

BULL has progressed from an execution-governance design to a release candidate with recorded host, guest, collector, packaging, and recovery behavior. The implementation ties authorization to exact operations, keeps admitted data separate from trusted runtime code, transports control and evidence through authenticated channels, and consumes approval before a protected effect. The latest integrated record reports 489 passing tests and 21 subtests, five passing real-KVM cases, and accepted external host and guest checkpoints. The historical governed-agent run adds bounded evidence for fixed read-only tools and controlled recovery.

The KB2040 result is part of that engineering record: software verification and replay protection are implemented, while the physical signing prerequisite is visibly absent and protected approval remains blocked. The next stage can therefore start from a precise build, a defined request protocol, and reproducible acceptance procedures. Each additional adapter or hardware capability must carry the same connection from authority to effect to evidence.

### AUTHORSHIP AND SOURCE NOTE

Luis Minier / Anharmonyclabs is the project author and a stakeholder in the reported results. ChatGPT assisted with

manuscript editing, layout, literature verification, and synthesis of repository evidence. Measurements are attributed to the retained project records. Independent research and primary technical sources are cited at the claims they support. Repository references are pinned to publication snapshot 437de3f; the records themselves identify their tested revisions.

## REFERENCES

- [1] J. H. Saltzer and M. D. Schroeder, *The Protection of Information in Computer Systems. Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [2] E. Debenedetti et al., *Defeating Prompt Injections by Design. arXiv:2503.18813v1*, 2025.
- [3] Anharmonyclabs, *Minier, BULL first technical paper, version 0.2, September 20, 2026. Paper I, v0.2*, 2026.
- [4] Anharmonyclabs, *Integrated validation data for clean candidate 603365e. site/data/validation/603365e.json*, 2026.
- [5] Anharmonyclabs, *Validation recorded September 23, including retained failures. docs/VALIDATION\_20260923.md*, 2026.
- [6] Anharmonyclabs, *Branch integration and current adapter/hardware integration scope. docs/BRANCH\_INTEGRATION.md*, 2026.
- [7] Anharmonyclabs, *Security claims, effect-based acceptance and trust limits. docs/SECURITY\_CLAIMS.md*, 2026.
- [8] Anharmonyclabs, *Production dispatcher/runtime and final broker-effect checks. src/bulldog/profiles.py*, 2026.
- [9] National Institute of Standards and Technology, *Standards for Security Categorization of Federal Information and Information Systems. FIPS PUB 199, doi:10.6028/NIST.FIPS.199, 2004*.
- [10] Anharmonyclabs, *Hardware authority design, bring-up, wiring and diagnostic limitations. docs/HARDWARE\_AUTHORITY.md*, 2026.
- [11] Anharmonyclabs, *Original technical drawings in the BULL repository. site/assets/architecture*, 2026.
- [12] Anharmonyclabs, *Historical MicroVM report, measured intervals and source identity. docs/MICROVM\_INTEGRATION\_REPORT.md*, 2026.
- [13] Anharmonyclabs, *Python package, version, dependencies and installation entry point. pyproject.toml*, 2026.
- [14] The Linux kernel development community, *Seccomp BPF (SECure COMputing with filters). Linux kernel documentation; accessed Sep. 23, 2026*.
- [15] M. Salaün and the Linux kernel development community, *Landlock: unprivileged access control. Linux kernel documentation; accessed Sep. 23, 2026*.
- [16] The Linux kernel development community, *Control Group v2. Linux kernel documentation; delegation and CPU, memory, and PID controllers; accessed Sep. 23, 2026*.
- [17] Anharmonyclabs, *Parent-validated sandbox attestation and pre-exec release. src/bulldog/namespace\_sandbox.py*, 2026.
- [18] Anharmonyclabs, *Reproducible deployment with independent operator authority. docs/REPRODUCIBLE\_DEPLOYMENT.md*, 2026.
- [19] The QEMU Project, *microvm virtual platform. QEMU system-emulation documentation; accessed Sep. 23, 2026*.
- [20] Anharmonyclabs, *MicroVM architecture, one-shot runner and local reproduction. microvm/README.md*, 2026.
- [21] Anharmonyclabs, *Bounded authenticated session control frames. src/bulldog/microvm\_protocol.py*, 2026.
- [22] Anharmonyclabs, *HTTPS checkpoint, guest relay and acknowledgement implementation. src/bulldog/audit\_transport.py*, 2026.
- [23] Anharmonyclabs, *Cloudflare Worker and D1 collector deployment. deploy/cloudflare-audit/README.md*, 2026.
- [24] H. Krawczyk, M. Bellare, and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication. RFC 2104, doi:10.17487/RFC2104, 1997*.
- [25] Anharmonyclabs, *September 20 external audit recovery evidence. docs/PRODUCTION\_EXTERNAL\_AUDIT\_EVIDENCE\_20260920.md*, 2026.
- [26] Anharmonyclabs, *Historical governed-agent evidence and exact source-manifest links. docs/evidence/governed-agent-20260922/README.md*, 2026.
- [27] Anharmonyclabs, *Consequential broker approval and OpenSSH ceremony contract. docs/HUMAN\_APPROVAL.md*, 2026.
- [28] Anharmonyclabs, *Durable request consumption, counters and audit ordering. src/bulldog/approval.py*, 2026.
- [29] SQLite developers, *Atomic Commit In SQLite. SQLite technical documentation; accessed Sep. 23, 2026*.
- [30] The OpenSSH Project, *ssh-keygen(1): OpenSSH authentication key utility. OpenBSD manual; FIDO AUTHENTICATOR section; accessed Sep. 23, 2026*.
- [31] Anharmonyclabs, *BULL-AUTH-1 fixed-width request and assertion encoding. src/bulldog/hardware\_approval/protocol.py*, 2026.
- [32] FIDO Alliance, *Client to Authenticator Protocol (CTAP), version 2.1. Proposed Standard, June 15, 2021*.
- [33] K. Rembor, Adafruit Industries, *Adafruit KB2040: Pinouts. Adafruit Learning System, published December 2021; revised January 2025; accessed Sep. 23, 2026*.
- [34] Anharmonyclabs, *Supported execution adapters and controlled recovery scope. docs/ADAPTER\_VALIDATION.md*, 2026.
- [35] Anharmonyclabs, *September 22 validation history, cgroups and optimization correction. docs/VALIDATION\_20260922.md*, 2026.
- [36] Anharmonyclabs, *Controlled storage, worker and collector recovery tests. tests/test\_recovery\_failures.py*, 2026.
- [37] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley Professional, 2002*.
- [38] Anharmonyclabs, *Three finite TLA+ specifications and configurations. formal/TLA*, 2026.
- [39] Anharmonyclabs, *September 20 benchmark method, CSV data and measured results. docs/BENCHMARK\_20260920.md*, 2026.
- [40] Anharmonyclabs, *Earlier candidate 428db9c machine-readable validation. docs/evidence/candidate-428db9c.json*, 2026.
- [41] Anharmonyclabs, *Local-model acceptance procedure and recovery rules. docs/GOVERNED\_AGENT\_RUN.md*, 2026.
- [42] Anharmonyclabs, *Source/binary distribution and guest source-materials release. docs/DISTRIBUTION.md*, 2026.
- [43] A. Agache et al., *Firecracker: Lightweight Virtualization for Serverless Applications. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 419–434, 2020*.
- [44] B. Schneier and J. Kelsey, *Secure Audit Logs to Support Computer Forensics. ACM Transactions on Information and System Security, 1999*.
- [45] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track, 2024*.